

1. fork() [System Call]

```
#include<stdio.h>
#include<unistd.h>
int main(){
    if(fork()==0)
        printf("Child\n");
    else
        printf("Parent\n");
    printf("PID=%d",getpid());
}
```

2. FCFS Scheduling

```
// Input: n, then each line: AT BT
#include<stdio.h>
int main(){
    int n,i; scanf("%d",&n);
    int AT[n],BT[n],CT[n],TAT[n],WT[n];
    for(i=0;i<n;i++){
        scanf("%d%d",&AT[i],&BT[i]);
        CT[i]=AT[i]+BT[i];
        for(j=0;j<n;j++){
            if(CT[i-1]<AT[j]) CT[j]=AT[j]+BT[j];
            else CT[j]=CT[i-1]+BT[j];
        }
        float tt=0,tw=0;
        for(i=0;i<n;i++){
            TAT[i]=CT[i]-AT[i]; WT[i]=TAT[i]-BT[i];
            tt+=TAT[i]; tw+=WT[i];
        }
        printf("Avg TAT=%.2f\n",tt/n);
        printf("Avg WT=%.2f",tw/n);
    }
}
```

3. SJF Scheduling (Non-Preemptive)

```
#include<stdio.h>
int main(){
    int n,i,time=0,c=0,min; scanf("%d",&n);
    int AT[n],BT[n],CT[n],TAT[n],WT[n],v[n];
    for(i=0;i<n;i++){
        scanf("%d%d",&AT[i],&BT[i]); v[i]=0;
    }
    while(c<n){
        min=-1;
        for(i=0;i<n;i++){
            if(AT[i]<=time && v[i]==0)
                if(min== -1 || BT[i]<BT[min]) min=i;
            if(min== -1) time++;
        }
        else{ time+=BT[min]; CT[min]=time; v[min]=1; c++; }
    }
    float tt=0,tw=0;
    for(i=0;i<n;i++){
        TAT[i]=CT[i]-AT[i]; WT[i]=TAT[i]-BT[i];
        tt+=TAT[i]; tw+=WT[i];
    }
    printf("Avg TAT=%.2f\n",tt/n);
    printf("Avg WT=%.2f",tw/n);
}
```

4. Round Robin Scheduling

```
// Input: n tq, then each: AT BT
#include<stdio.h>
int main(){
    int n,tq,i,time=0,c=0;
    scanf("%d%d",&n,&tq);
    int AT[n],BT[n],RT[n],CT[n],TAT[n],WT[n];
    for(i=0;i<n;i++){
        scanf("%d%d",&AT[i],&BT[i]); RT[i]=BT[i];
    }
    while(c<n){
        int done=1;
        for(i=0;i<n;i++){
            if(RT[i]>0 && AT[i]<=time){
                done=0;
                if(RT[i]>tq){ time+=tq; RT[i]-=tq; }
                else{ time+=RT[i]; CT[i]=time; RT[i]=0; c++; }
            }
            if(done) time++;
        }
        float tt=0,tw=0;
        for(i=0;i<n;i++){
            TAT[i]=CT[i]-AT[i]; WT[i]=TAT[i]-BT[i];
            tt+=TAT[i]; tw+=WT[i];
        }
        printf("Avg TAT=%.2f\n",tt/n);
        printf("Avg WT=%.2f",tw/n);
    }
}
```

5. Priority Scheduling (Non-Preemptive)

```
// Input: n, then each: AT BT Priority(lower=higher)
#include<stdio.h>
int main(){
    int n,i,time=0,c=0,min; scanf("%d",&n);
    int AT[n],BT[n],P[n],CT[n],TAT[n],WT[n],v[n];
    for(i=0;i<n;i++){
        scanf("%d%d%d",&AT[i],&BT[i],&P[i]); v[i]=0;
    }
    while(c<n){
        min=-1;
        for(i=0;i<n;i++){
            if(AT[i]<=time && v[i]==0)
                if(min== -1 || P[i]<P[min]) min=i;
            if(min== -1) time++;
        }
        else{ time+=BT[min]; CT[min]=time; v[min]=1; c++; }
    }
    float tt=0,tw=0;
    for(i=0;i<n;i++){
        TAT[i]=CT[i]-AT[i]; WT[i]=TAT[i]-BT[i];
        tt+=TAT[i]; tw+=WT[i];
    }
    printf("Avg TAT=%.2f\n",tt/n);
    printf("Avg WT=%.2f",tw/n);
}
```

6. Banker's Algorithm

```
// Input: p r | Alloc matrix | Max matrix | Available[]
#include<stdio.h>
int main(){
    int p,r,i,j,k; scanf("%d",&p,&r);
    int a[p][r],m[p][r],n[p][r];
    int av[r],w[r],f[p],s[p];
    for(i=0;i<p;i++){ // Allocation
        for(j=0;j<r;j++) scanf("%d",&a[i][j]);
    }
    for(i=0;i<p;i++){ // Max
        for(j=0;j<r;j++) scanf("%d",&m[i][j]);
    }
    for(i=0;i<r;i++){ scanf("%d",&av[i]); w[i]=av[i]; }
    for(i=0;i<p;i++){
        for(j=0;j<r;j++) n[i][j]=m[i][j]-a[i][j];
    }
    for(i=0;i<p;i++) f[i]=0;
    int c=0;
    while(c<p){
        int found=0;
        for(i=0;i<p;i++){
            if(f[i]==0){
                int ok=1;
                for(j=0;j<r;j++){
                    if(n[i][j]>w[j]){ ok=0; break; }
                }
                if(ok){
                    for(k=0;k<r;k++) w[k]+=a[i][k];
                    s[c++]=i; f[i]=1; found=1;
                }
            }
        }
        if(!found) break;
    }
    if(c==p){
        printf("Safe: ");
        for(i=0;i<p;i++) printf("P%d ",s[i]);
    } else printf("Unsafe");
}
```

7. Shared Memory

```
#include<stdio.h>
#include<sys/ipc.h>
#include<sys/shm.h>
int main(){
    int id=shmget(1,50,0666|IPC_CREAT);
    char *p=shmat(id,NULL,0);
    scanf("%s",p);
    printf("Data=%s",p);
}
```

8. Producer-Consumer (Semaphore)

```
#include<stdio.h>
#include<stdlib.h>
int mutex=1,full=0,empty=5,x=0;
int wait(int s){ return --s; }
int signal(int s){ return ++s; }
void producer(){
    mutex=wait(mutex); empty=wait(empty);
    full=signal(full);
    x++; printf("Produced %d\n",x);
    mutex=signal(mutex);
}
void consumer(){
    mutex=wait(mutex); full=wait(full);
    empty=signal(empty);
    printf("Consumed %d\n",x); x--;
    mutex=signal(mutex);
}
int main(){
    int n;
    while(1){
        printf("1.Producer 2.Consumer 3.Exit\n");
        scanf("%d",&n);
        switch(n){
            case 1:
                if(mutex==1&&empty!=0) producer();
                else printf("Buffer Full\n"); break;
            case 2:
                if(mutex==1&&full!=0) consumer();
                else printf("Buffer Empty\n"); break;
            case 3: exit(0);
        }
    }
}
```

9. First Fit Memory Allocation

```
// Input: nb np | block sizes | process sizes
#include<stdio.h>
int main(){
    int b[10],p[10],nb,np,i,j;
    scanf("%d%d",&nb,&np);
    for(i=0;i<nb;i++) scanf("%d",&b[i]);
    for(i=0;i<np;i++) scanf("%d",&p[i]);
    for(i=0;i<np;i++){
        for(j=0;j<nb;j++){
            if(b[j]>=p[i]){
                printf("P%d->B%d\n",i+1,j+1);
                b[j]-=p[i]; break;
            }
        }
    }
}
```

10. Best Fit Memory Allocation

```
#include<stdio.h>
int main(){
    int b[10],p[10],nb,np,i,j,k;
    scanf("%d%d",&nb,&np);
    for(i=0;i<nb;i++) scanf("%d",&b[i]);
    for(i=0;i<np;i++) scanf("%d",&p[i]);
    for(i=0;i<np;i++){
        k=-1;
        for(j=0;j<nb;j++){
            if(b[j]>=p[i]&&(k==-1||b[j]<b[k])) k=j;
        }
        if(k!=-1){
            printf("P%d->B%d\n",i+1,k+1);
            b[k]-=p[i];
        }
    }
}
```

11. FIFO Page Replacement

```
// Input: n(pages) m(frames) | reference string
#include<stdio.h>
int main(){
    int p[20],f[10],n,m,i,j,k=0,c=0,x;
    scanf("%d%d",&n,&m);
    for(i=0;i<n;i++) scanf("%d",&p[i]);
    for(i=0;i<m;i++) f[i]=-1;
    for(i=0;i<n;i++){
        x=0;
        for(j=0;j<m;j++){
            if(f[j]==p[i]) x=1;
            if(!x){ f[k]=p[i]; k=(k+1)%m; c++; }
        }
    }
    printf("Faults=%d",c);
}
```

12. LRU Page Replacement

```
#include<stdio.h>
int main(){
    int p[20],f[10],t[10]={0};
    int n,m,i,j,c=0,x,mn,tm=0;
    scanf("%d%d",&n,&m);
    for(i=0;i<n;i++) scanf("%d",&p[i]);
    for(i=0;i<m;i++) f[i]=-1;
    for(i=0;i<n;i++){
        tm++; x=0;
        for(j=0;j<m;j++){
            if(f[j]==p[i]){ t[j]=tm; x=1; }
        }
        if(!x){
            mn=0;
            for(j=1;j<m;j++){
                if(t[j]<t[mn]) mn=j;
            }
            f[mn]=p[i]; t[mn]=tm; c++;
        }
    }
    printf("Faults=%d",c);
}
```

13. Shell Scripts

```
# --- Addition ---
#!/bin/bash
read a b
echo $((a+b))

# --- Leap Year ---
#!/bin/bash
read n
if [ $(n%4) -eq 0 ]; then
    echo "Leap Year"
else
    echo "Not Leap Year"
fi

# --- Reverse Number ---
#!/bin/bash
read n; rev=0
while [ $n -gt 0 ]; do
    r=$((n%10))
    rev=$((rev*10+r))
    n=$((n/10))
done
echo $rev

# --- Fibonacci (first n terms) ---
#!/bin/bash
read n; a=0; b=1
echo $a; echo $b
for((i=3;i<=n;i++)); do
    c=$((a+b)); echo $c
    a=$b; b=$c
done
```